PEP 668 – Marking Python base environments as “externally managed”

Important

This PEP is a historical document. The up-to-date, canonical spec, Externally Managed Environments, is maintained on the PyPA specs page.

See the PyPA specification update process for how to propose changes.

Abstract

A long-standing practical problem for Python users has been conflicts between OS package managers and Python-specific package management tools like pip. These conflicts include both Python-level API incompatibilities and conflicts over file ownership.

Historically, Python-specific package management tools have defaulted to installing packages into an implicit global context. With the standardization and popularity of virtual environments, a better solution for most (but not all) use cases is to use Python-specific package management tools only within a virtual environment.

This PEP proposes a mechanism for a Python installation to communicate to tools like pip that its global package installation context is managed by some means external to Python, such as an OS package manager. It specifies that Python-specific package management tools should neither install nor remove packages into the interpreter’s global context, by default, and should instead guide the end user towards using a virtual environment.

It also standardizes an interpretation of the sysconfig schemes so that, if a Python-specific package manager is about to install a package in an interpreter-wide context, it can do so in a manner that will avoid conflicting with the external package manager and reduces the risk of breaking software shipped by the external package manager.

Terminology

A few terms used in this PEP have multiple meanings in the contexts that it spans. For clarity, this PEP uses the following terms in specific ways:

Motivation

Thanks to Python’s immense popularity, software distros (by which we mean Linux and other OS distros as well as overlay distros like Homebrew and MacPorts) generally ship Python for

two purposes: as a software package to be used in its own right by end users, and as a language dependency for other software in the distro.

For example, Fedora and Debian (and their downstream distros, as well as many others) ship a /usr/bin/python3 binary which provides the python3 command available to end users as well as the #!/usr/bin/python3 shebang for Python-language software included in the distro. Because there are no official binary releases of Python for Linux/UNIX, almost all Python end users on these OSes use the Python interpreter built and shipped with their distro.

The python3 executable available to the users of the distro and the python3 executable available as a dependency for other software in the distro are typically the same binary. This means that if an end user installs a Python package using a tool like pip outside the context of a virtual environment, that package is visible to Python-language software shipped by the distro. If the newly-installed package (or one of its dependencies) is a newer, backwards-incompatible version of a package that was installed through the distro, it may break software shipped by the distro.

This may pose a critical problem for the integrity of distros, which often have package-management tools that are themselves written in Python. For example, it's possible to unintentionally break Fedora's dnf command with a pip install command, making it hard to recover.

This applies both to system-wide installs (sudo pip install) as well as user home directory installs (pip install --user), since packages in either location show up on the sys.path of /usr/bin/python3.

There is a worse problem with system-wide installs: if you attempt to recover from this situation with sudo pip uninstall, you may end up removing packages that are shipped by the system's package manager. In fact, this can even happen if you simply upgrade a package - pip will try to remove the old version of the package, as shipped by the OS. At this point it may not be possible to recover the system to a consistent state using just the software remaining on the system.

Over the past many years, a consensus has emerged that the best way to install Python libraries or applications (when not using a distro's package) is to use a virtual environment. This approach was popularized by the PyPA virtualenv project, and a simple version of that approach is now available in the Python standard library as venv. Installing a Python package into a virtualenv prevents it from being visible to the unqualified /usr/bin/python3 interpreter and prevents breaking system software.

In some cases, however, it's useful and intentional to install a Python package from outside of the distro that influences the behavior of distro-shipped commands. This is common in the case of software like Sphinx or Ansible which have a mechanism for writing Python-language extensions. A user may want to use their distro's version of the base software (for reasons of paid support or security updates) but install a small extension from PyPI, and they'd want that extension to be importable by the software in their base system.

While this continues to carry the risk of installing a newer version of a dependency than the operating system expects or otherwise negatively affecting the behavior of an application, it does not need to carry the risk of removing files from the operating system. A tool like pip should be able to install packages in some directory on the default sys.path, if specifically requested, without deleting files owned by the system's package manager.

Therefore, this PEP proposes two things.

First, it proposes a way for distributors of a Python interpreter to mark that interpreter as having its packages managed by means external to Python, such that Python-specific tools like pip should not change the installed packages in the interpreter's global sys.path in any way (add, upgrade/downgrade, or remove) unless specifically overridden. It also provides a means for the distributor to indicate how to use a virtual environment as an alternative.

This is an opt-in mechanism: by default, the Python interpreter compiled from upstream sources will not be so marked, and so running pip install with a self-compiled interpreter, or with a distro that has not explicitly marked its interpreter, will work as it always has worked.

Second, it sets the rule that when installing packages to an interpreter's global context (either to an unmarked interpreter, or if overriding the marking), Python-specific package managers should modify or delete files only within the directories of the sysconfig scheme in which they would create files. This permits a distributor of a Python interpreter to set up two directories, one for its own managed packages, and one for unmanaged packages installed by the end user, and ensure that installing unmanaged packages will not delete (or overwrite) files owned by the external package manager.

Rationale

As described in detail in the next section, the first behavior change involves creating a marker file named EXTERNALLY-MANAGED, whose presence indicates that non-virtual-environment package installations are managed by some means external to Python, such as a distro's package manager. This file is specified to live in the stdlib directory in the default sysconfig scheme, which marks the interpreter / installation as a whole, not a particular location on sys.path. The reason for this is that, as identified above, there are two related problems that risk breaking an externally-managed Python: you can install an incompatible new version of a package system-wide (e.g., with sudo pip install), and you can install one in your user account alone, but in a location that is on the standard Python command's sys.path (e.g., with pip install --user). If the marker file were in the system-wide site-packages directory, it would not clearly apply to the second case. The Alternatives section has further discussion of possible locations.

The second behavior change takes advantage of the existing sysconfig setup in distros that have already encountered this class of problem, and specifically addresses the problem of a Python-specific package manager deleting or overwriting files that are owned by an external package manager.

Use cases

The changed behavior in this PEP is intended to “do the right thing” for as many use cases as possible. In this section, we consider the changes specified by this PEP for several representative use cases / contexts. Specifically, we ask about the two behaviors that could be changed by this PEP:

1. Will a Python-specific installer tool like pip install permit installations by default, after implementation of this PEP?
2. If you do run such a tool, should it be willing to delete packages shipped by the external (non-Python-specific) package manager for that context, such as a distro package manager?

(For simplicity, this section discusses pip as the Python-specific installer tool, though the analysis should apply equally to any other Python-specific package management tool.)

This table summarizes the use cases discussed in detail below:

Case	**Description**	**pip install permitted**	**Deleting externally-installed packages permitted**
1	Unpatched CPython	Currently yes; stays yes	Currently yes; stays yes
2	Distro /usr/bin/python3	Currently yes; becomes no (assuming the distro adds a marker file)	Currently yes (except on Debian); becomes no
3	Distro Python in venv	Currently yes; stays yes	There are no externally-installed packages
4	Distro Python in venv with --system-site-packages	Currently yes; stays yes	Currently no; stays no
5	Distro Python in Docker	Currently yes; becomes no (assuming the distro adds a marker file)	Currently yes; becomes no
6	Conda environment	Currently yes; stays yes	Currently yes; stays yes
7	Dev-facing distro	Currently yes; becomes no (assuming they add a marker file)	Currently often yes; becomes no (assuming they configure sysconfig as needed)
8	Distro building packages	Currently yes; can stay yes	Currently yes; becomes no

Case	Description	pip install permitted	Deleting externally-installed packages permitted
9	PYTHONHOME copied from a distro Python stdlib	Currently yes; becomes no	Currently yes; becomes no
10	PYTHONHOME copied from upstream Python stdlib	Currently yes; stays yes	Currently yes; stays yes

In more detail, the use cases above are:

1. If there is a concept of “externally-installed” packages in this case, it’s something outside the OS and generally managed by whoever built and installed this CPython. Because the installer chose not to add a marker file or modify sysconfig schemes, they’re choosing the current behavior, and pip install can remove any packages available in this CPython.
2. (We have seen user reports of externally-installed packages being deleted on Debian or a derivative. We suspect this is because the user has previously run sudo pip install --upgrade pip and therefore now has a version of /usr/bin/pip without the Debian patch; standardizing this behavior in upstream package installers would address this problem.)
3. Inside a virtual environment, all packages are owned by that environment. Even when pip, setuptools, etc. are installed into the environment, they are and should be managed by tools specific to that environment; they are not system-managed.
4. Currently, the answer to “Will pip delete externally-installed packages” is no, because pip has a special case for running in a virtual environment and attempting to delete packages outside it. After this PEP, the answer remains no, but the reasoning becomes more general: system site packages will be outside any of the sysconfig schemes used for package management in the environment.
5. It is the view of the authors of this PEP that it’s still a good idea to use virtual environments with distro-installed Python interpreters, even in single-application container images. Even though they run a single application, that application may run commands from the OS that are implemented in Python, and if you’ve installed or upgraded the distro-shipped Python packages using Python-specific tools, those commands may break.
6. We don’t believe this PEP requires any changes to Conda, and versions of pip that have implemented the changes in this PEP will continue to behave as they currently do inside Conda environments. (That said, it may be worth considering whether to use separate sysconfig schemes for pip-installed and Conda-installed software, for the same reasons it’s a good idea for other distros.)
7. If the distro supports/encourages creating a virtual environment from the distro’s Python interpreter, there may also be custom instructions for how to properly set up a virtual environment (as for example Nixpkgs does).

8. As a result of this PEP, pip will no longer be able to remove packages already on the system. However, this behavior change is fine because a package build process should not (and generally cannot) include instructions to delete some other files on the system; it can only package up its own files.
9. This behavior change seems to be defensible, in that if your PYTHONHOME is a straight copy of the distro's Python, it should behave like the distro's Python.
10. Because the behavior changes in this PEP are keyed off of files in the standard library (the marker file in stdlib and the behavior of the sysconfig module), the behavior is just like an unmodified upstream CPython (case 1).

Specification

Marking an interpreter as using an external package manager

Before a Python-specific package installer (that is, a tool such as pip - not an external tool such as apt) installs a package into a certain Python context, it should make the following checks by default:

1. Is it running outside of a virtual environment? It can determine this by whether sys.prefix == sys.base_prefix (but see Backwards Compatibility).
2. Is there an EXTERNALLY-MANAGED file in the directory identified by sysconfig.get_path("stdlib", sysconfig.get_default_scheme())?

If both of these conditions are true, the installer should exit with an error message indicating that package installation into this Python interpreter's directory are disabled outside of a virtual environment.

The installer should have a way for the user to override these rules, such as a command-line flag --break-system-packages. This option should not be enabled by default and should carry some connotation that its use is risky.

The EXTERNALLY-MANAGED file is an INI-style metadata file intended to be parsable by the standard library configparser module. If the file can be parsed by configparser.ConfigParser(interpolation=None) using the UTF-8 encoding, and it contains a section [externally-managed], then the installer should look for an error message specified in the file and output it as part of its error. If the first element of the tuple returned by locale.getlocale(locale.LC_MESSAGES), i.e., the language code, is not None, it should look for the error message as the value of a key named Error- followed by the language code. If that key does not exist, and if the language code contains underscore or hyphen, it should look for a key named Error- followed by the portion of the language code before the underscore or hyphen. If it cannot find either of those, or if the language code is None, it should look for a key simply named Error.

If the installer cannot find an error message in the file (either because the file cannot be parsed or because no suitable error key exists), then the installer should just use a pre-

defined error message of its own, which should suggest that the user create a virtual environment to install packages.

Software distributors who have a non-Python-specific package manager that manages libraries in the sys.path of their Python package should, in general, ship a EXTERNALLY-MANAGED file in their standard library directory. For instance, Debian may ship a file in /usr/lib/python3.9/EXTERNALLY-MANAGED consisting of something like